

Lightweight Testing of Persistent Amortized Time Complexity in the Credit Monad

Anton Lorenzen

University of Edinburgh
Edinburgh, UK
anton.lorenzen@ed.ac.uk

Abstract

Persistent data structures are ubiquitous in functional programming languages and their designers frequently have to reason about amortized time complexity. But proving amortized bounds is difficult in a persistent setting, and pen-and-paper proofs give little assurance of correctness, while a full mechanization in a proof assistant can be too involved for the casual user. In this work, we define a strict domain specific language (DSL) for testing the amortized time complexity of persistent data structures using QuickCheck. Our DSL can give strong evidence of correctness, while imposing low overhead on the user. We have used our DSL to check the amortized time complexity of all lazy data structures in Okasaki's book. As a sign of our approach's effectiveness, we re-discovered a previously unreported gap in the analysis of persistent Finger Trees.

We have released our work as the [creditmonad](#) package.

CCS Concepts: • Theory of computation → Data structures design and analysis; • Software and its engineering → Functional languages.

Keywords: Laziness, Lazy Constructors, Defunctionalization, QuickCheck, Haskell, Amortized Complexity, Persistent Data Structures, Finger Trees

ACM Reference Format:

Anton Lorenzen. 2025. Lightweight Testing of Persistent Amortized Time Complexity in the Credit Monad. In *Proceedings of the 18th ACM SIGPLAN International Haskell Symposium (Haskell '25)*, October 12–18, 2025, Singapore, Singapore. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3759164.3759351>

1 Introduction

Purely functional data structures are persistent by design: programmers can keep working with old versions of their data structures at no extra cost. For example, consider a functional queue with a `snoc` operation to append an element

to the back of the queue and an `uncons` operation to remove an element from the front of the queue. We can use it like this:

```
let q = snoc (snoc (snoc empty 1) 2) 3
    a = foo q
    b = uncons q
in ...
```

Persistence allows us to easily reason about this code. Even without knowing the implementation of `foo`, we know that `b` will always be `Just (1, q')` where `q'` contains the elements `2` and `3`. This is a huge benefit, since it allows programmers to understand programs compositionally and to reason about them in a modular way.

However, this valuable property comes at a price: classic proofs of amortized time complexity only apply to *sequential* but not to *persistent* use-cases. For example, many queue implementations perform an expensive restructuring in the `uncons` operation which takes linear time in the worst case. One might try to argue that `uncons` still only takes *amortized* constant time, by accumulating *credit* during `snoc` operations which is spent during `uncons` operations. But this style of reasoning does not work in the persistent setting: if `foo` calls `uncons` on `q`, then the expensive restructuring is performed multiple times and so the credit is spent multiple times.

Okasaki [1998] showed how to prove amortized time complexity bounds in the persistent setting. This commonly involves two steps: first, the data structure is modified to strategically use lazy thunks to delay expensive operations. Then, operations like `snoc` pay off *debt* assigned to those thunks. If one can argue that the debt is paid off by the time the restructuring is needed, then the data structure will observe amortized bounds also in a persistent setting. Okasaki shows how to perform this reasoning for a wide range of functional data structures, including queues, heaps, random-access lists and sortable collections: his book has become an indispensable companion for the designer of functional data structures.

But to the casual user, it can often be hard to correctly perform this reasoning from scratch. It would easily be possible to make a mistake in an Okasaki-style pen-and-paper proof, or to get lost in the details of the reasoning. This is exacerbated by Okasaki's frequent rearrangement of debt between thunks which is justified by several distinct reasoning principles. Recently, mechanizations have received much interest [Danielsson 2008; Pottier et al. 2024; Xia et



This work is licensed under a Creative Commons Attribution 4.0 International License.

Haskell '25, Singapore, Singapore

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2147-2/25/10

<https://doi.org/10.1145/3759164.3759351>

al. 2024], but they are labor-intensive and only cover some of Okasaki’s data structures so far.

In this work, we bring the convenience of QuickCheck-based testing to persistent amortized time complexity. We present a strict domain specific language (DSL), which enables users to quickly check their reasoning by adding few annotations to an implementation of their data structure. While the DSL can not be used to *prove* the amortized time complexity of a persistent data structure, it allows us to check our reasoning against a range of examples, which gives us strong evidence of correctness.

Our DSL can also be useful in the design of new data structures, since it allows for rapid prototyping. If a certain line of reasoning is wrong, QuickCheck quickly comes up with counter-examples. Since the annotation burden is low, a design can easily be changed to accommodate different reasoning. Furthermore, if the user chooses to defunctionalize the lazy thunks [Lorenzen et al. 2025], our DSL can print lazy data structures during evaluation, which can yield insights that are hard to find using pen-and-paper proofs.

Debits or Credits? Diverging from sequential analysis of amortized time complexity, Okasaki relies on reasoning about *debt* instead of *credit*. Okasaki claims that this is necessary, since credits are insufficient in a persistent setting: “although savings can only be spent once, it does no harm to pay off debt more than once” [Okasaki 1998, Section 6.2]. This can be confusing to those more used to traditional methods: Kaplan et al. [2000] mention it as an explicit disadvantage of Okasaki’s method.

However, the reason that debt works in a persistent setting does not seem to be that it can be paid off multiple times, but rather that it is *placed on lazy thunks*. The runtime of functional languages guarantees that lazy thunks are only forced once (at least in a single-threaded setting), which implies that credits placed on a thunk are only spent once.

This makes it equally sound to reason about credits rather than debits. It is then just a superficial choice which one to use in a proof, since they can be easily converted to each other. Reasoning using debt typically has the advantage of shorter invariants, but an analysis using credits is more similar to the analysis of sequential data structures. To improve the user experience for casual users, we base our development on credits.

Which Reasoning Principle? Okasaki [1998] employs a variety of reasoning principles: the Bankers Method, Physicist Method, debit inheritance, left-linear debit invariants and debit passing. While this shows the power of his approach, it presents a didactic issue: to be easy to use, a library should ideally rely only on one reasoning principle.

Danielsson [2008] identifies a variant of Okasaki’s debit-passing technique as most promising, but also notes its limitation: “This version is arguably easier to explain [than Okasaki’s Bankers Method], but it is less general, so fewer

programs can be properly analysed.” Due to its simplicity, we have chosen to base our DSL on his principle: following our credit-based style, we call this principle *credit passing*.

In our work, we have discovered that credit passing is actually quite powerful and can be used to analyse almost all data structures described by Okasaki [1998]. This is a surprising result! In most cases, our implementations could be adopted directly from Okasaki’s book, but for some data structures like Binomial Heaps or the Bootstrapped Queue, we needed additional insights.

In fact, credit passing only fails for three data structures of Okasaki: the Bankers Queue, the Bankers Deque, and Catenable Lists. While the last data structure is challenging to analyse, we can extend our analysis to the first two. To achieve this, we describe *credit inheritance* as a novel extension of the credit passing technique. While credit inheritance is inspired by Okasaki’s debit inheritance, it has the advantage of allowing us to reason without considering future thunks.

1.1 Contributions

In this work, we bring the convenience of QuickCheck-based testing to persistent data structures. In particular,

- We introduce a QuickCheck-based framework for testing the amortized time complexity of data structures (Section 2).
- We propose an ergonomic way to embed laziness and, in particular, lazy constructors in a DSL (Section 3 and 4).
- We define the Credit Monad for reasoning about persistent amortized time complexity using credit-passing style and demonstrate it by analysing Okasaki’s Implicit Queue (Section 5).
- We propose the novel technique of Credit Inheritance and use it to analyse Okasaki’s Bankers Queue (Section 6).
- As a case study, we analyse the amortized time complexity of Finger Trees. We discuss a gap in the classic analysis of measurement and extend Claessen [2020]’s analysis to the persistent setting (Section 7).

2 Testing (Amortized) Time Complexity

To reason about the cost of a computation, a classic approach is to denote it in a monad which can count the number of steps taken by the computation [Gibbons and Hinze 2011]:

```
class Monad m => MonadCount m where
  tick :: m ()
```

To see this in action, consider the following `rev` function, which reverses the list `xs` and appends it to the list `acc`:

```
rev :: MonadCount m => [a] -> [a] -> m [a]
rev [] acc = pure acc
rev (x : xs) acc = tick >> rev xs (x : acc)
```

If `xs` has length n , then `rev xs acc` takes n steps to finish. Note that a formal machine model might require slightly more

steps in practice, say, $2n$ to account for both the deconstruction of `xs` and the construction of the new list. However, by placing a call to `tick` at each recursive call of `rev`, we ensure that any machine model will require only a constant factor more steps and so our result will be asymptotically correct.

The `rev` function is easy to analyse since there is no hidden state: it always takes exactly n steps. But analysing a data structure is not always so easy. Consider the following simple implementation of a first-in-first-out queue [Okasaki 1998]:

```
data Batched a m = Batched [a] [a]

snoc :: MonadCount m => Batched a m -> a -> m (Batched a m)
snoc (Batched front rear) x = batched front (x:rear)
uncons :: MonadCount m
=> Batched a m -> m (Maybe (a, Batched a m))
uncons (Batched [] rear) = pure Nothing
uncons (Batched (x:front) rear) = do
  q' <- batched front rear
  pure $ Just (x, q')
```

The `Batched a m` type consists of two lists `front` and `rear`; the `m` parameter can be ignored for now. The `snoc` operation pushes an element to the back of the queue and the `uncons` operation pops an element from the front of the queue. Both of these operations can be implemented in terms of cons-ing to a list. The real magic lies in the smart constructor `batched`, which reverses the `rear` list whenever the `front` list is empty:

```
batched :: MonadCount m => [a] -> [a] -> m (Batched a m)
batched [] rear = do
  front <- rev rear []
  pure $ Batched front []
batched front rear = pure $ Batched front rear
```

A simple worst-case analysis of this data structure works like this: if there are n elements in the queue, they might all be in the `rear` list. Then `batched` calls `tick` n times when reversing the `rear` list which implies that `uncons` takes $O(n)$ worst-case time.

2.1 Amortized Analysis

To see how many steps the queue actually takes, we can implement the `MonadCount` interface using the `CounterM` monad which tracks the number of ticks as a state. For example, we can add ten elements to the queue and successively remove them:

```
> runCounterM $ empty >= flip (foldM snoc) [1..10]
>= unfoldM uncons
Right ([1,2,3,4,5,6,7,8,9,10], Ticks 10)
```

In this computation, only 10 ticks have been executed. This is surprising, since our worst-case analysis would suggest that n `uncons` operations take $O(n^2)$ time – but in practice they seem to take only $O(n)$ time.

This suggests a far better *amortized* analysis: since each element can transition from the `rear` list to the `front` list at most once, and doing so takes a single step, the total cost of n `snoc` and `uncons` operations is only n . While all of those n steps can happen during a single `uncons` operation, such an expensive operation is preceded by n constant-time `snoc` operations and followed by n constant-time `uncons` operations.

In other words, the amortized time complexity of `snoc` and `uncons` is constant.

2.2 Testing Amortized Time Complexity

We can test this hypothesis using QuickCheck [Claessen and Hughes 2000]. We generate random queue operations using the data type `QueueOp`. In the `Arbitrary` instance, we bias towards `snoc` operations to ensure that our queues grow in size:

```
data QueueOp a = Snoc a | Uncons
deriving (Eq, Ord, Show)

instance Arbitrary a => Arbitrary (QueueOp a) where
  arbitrary = frequency
    [ (7, Snoc <$> arbitrary), (3, pure Uncons) ]
```

To test the data structure, we need to connect the operations to the queue type. Our library defines the `DataStructure` type class for this purpose: it allows us to create an instance of the data structure `t m` and to perform an operation `op` with a given cost on the data structure. `MonadLazy` and `MonadInherit` are subclasses of `MonadCount` and will be defined later.

```
class (Arbitrary op, Show op)
=> DataStructure t op | t -> op where
  create :: MonadLazy m => m (t m)
  perform :: MonadInherit m
=> Size -> t m -> op -> m (Size, t m)
  cost :: Size -> op -> Credit
```

Our library internally tracks the `Size` of the data structure, which is recomputed when an operation is performed. We may need to inspect the internal state of the data structure to determine the new size, since operations like inserting into a binary search tree only increase the size if the to-be-inserted element is not already present. In contrast, the cost function computes the amount of credits necessary to perform a given operation from the size of the data structure and the operation itself; without inspecting the internal state.

In the case of the `Batched a m` type, we can define the `DataStructure` instance as follows. Our cost bound assigns one credit to each `Snoc` operation, since it adds to the `rear` list, and no credits to `Uncons`, since it can use the existing credits to pay for the restructuring.

```
instance (Arbitrary a, Show a)
=> DataStructure (Batched a) (QueueOp a) where
  create = pure $ Batched [] []
  perform sz q (Snoc x) = (sz + 1,) <$> snoc q x
  perform sz q Uncons = do
    m <- uncons q
    case m of
      Nothing -> pure (sz, Batched [] [])
      Just (_, q') -> pure (sz - 1, q')
  cost n (Snoc _) = 1
  cost n Uncons = 0
```

Using this definition, our framework allows us to test the amortized time complexity by running a large number of random operations on the queue. We check that the total cost of the operations remains within the stated bounds:

```
> quickCheck $ checkCredits @(Batched Int) Path
+++ OK, passed 100 tests (0% persistent).
```

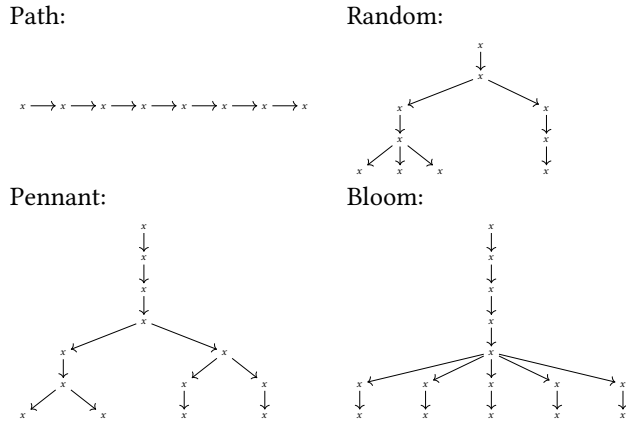


Fig. 1. Examples for the execution traces we test.

2.3 Modelling Persistence

The above amortized analysis of the Batched Queue only works for *sequential* use-cases. By invoking `checkCreditsTrace` with the `Path` parameter, we are instructing it to only test lists of operations, where each operation is performed on the queue generated by the previous operation.

But in a functional language we can use queues persistently. This allows users to keep a reference to an old version of the queue and to perform operations on that version. We can model this using an *execution trace* [Okasaki 1998, Section 6.1]: a tree where each node corresponds to an operation on the queue. If a node has several children, this corresponds to the user saving a reference and invoking the operations of the children on the same version of the queue.

In the example above, the `Path` execution trace is a tree where each node has at most one child: such a list-like execution trace models sequential use-cases. To test persistent use-cases, we can invoke `checkCreditsTrace` with a type of execution trace where some nodes have multiple children.

2.4 Generating Execution Traces

To accurately report faulty reasoning, it is important to test a variety of execution traces. We have implemented four different generators for trees to test our data structures:

- **Path**: a path with no persistence.
- **Random**: a tree using the default generator.
- **Pennant**: a tree which splits at about half its depth into two subtrees which are itself pennants.
- **Bloom**: a tree which splits into many subtrees close to the bottom.

In practice, the `Bloom` generator is particularly good for finding counter-examples, and it matches the typical shape of shrunk counter-examples produced by the other shapes. In contrast, the default `Random` generator is much less useful, since it generates trees of insufficient depth.

We show idealised examples of the execution traces in Figure 1. To print realistic examples, we can write (using `Data.Tree` and `Test.QuickCheck`):

```
> putStrLn . drawTree . fmap show
  =<< generate (genExecutionTrace @(QueueOp Int) Path)
```

2.5 The Batched Queue Revisited

If we invoke QuickCheck with a persistent strategy, we can easily find a counter-example:¹

```
> quickCheck $ checkCreditsTrace @(Batched Int) Random
Batched Queue (random):
*** Failed! Falsified (after 23 tests and 9 shrinks):
Snoc (-9): Queue([-9], [])
|
|  Snoc (-5): Queue([-9], [-5])
|  +- Uncons: Queue([-5], [])
|  |
|  +- Uncons: Out of credits for cell 0
```

After adding two elements to the back of the queue, we have used one credit and may use one more. This is where persistence comes into play: we perform two operations on *the same version of* the queue. The `uncons` operations both use up one credit and so we run out of credits. The reason why our earlier analysis failed is that we have duplicated work: an element can transition from the rear list to the front list more than once if we create multiple references to the queue where it is in the rear list.

For the Batched Queue, which uses no laziness, we can only make the analysis work in a persistent setting by providing the `uncons` operations with n credits. In fact, if we do so, QuickCheck accepts our new analysis. But this bound is as bad as our initial worst-case analysis and so we gain no benefit from amortization. This is the central insight motivating Okasaki’s work on persistent data structures: classic amortized analysis fails in a persistent setting. Okasaki [1998] instead proposes new variants of known data structures that cleverly use laziness and proposes new analysis techniques to show amortized bounds in a persistent setting.

2.6 Limitations

QuickCheck does not always come up with a counter-example even if one exists. In the Batched Queue example above, we may try assigning only $2n/3$ credits to `uncons` instead of the necessary n credits. At the settings we use, QuickCheck will refute this reliably, but it will usually accept $3n/4$ as a valid bound. As such, our library is good at distinguishing between different asymptotic bounds, but not as good at checking precise constant factors.

Our library is currently not suitable for data structures that achieve persistence using internal mutation. Such data structures are often only *semi-persistent*, where the user may access previous versions of a data structure only by abandoning the current version; see Allain et al. [2024] for an example and a detailed discussion. Our library only generates

¹To create that output, the `checkCreditsTrace` function requires us to implement pretty-printing functionality which we omit here for brevity.

tests with this behaviour and is thus unable to distinguish between semi-persistent and truly persistent data structures.

This problem arises since our model is slightly too weak: while an execution trace specifies *which* operations are performed, it does not specify *in which order* the operations are performed. In fact, functional programs are free to perform the operations of an execution trace in any order, as long as parents are executed before their children. This suggests that our library should test all topological sorts of the execution trace. However, we currently evaluate each execution trace only via a pre-order traversal, which corresponds exactly to abandoning the current version of the data structure when going back to a previous version. Pottier [2021]’s testing library does not suffer from this limitation.

3 Laziness in a Strict DSL

To define data structures that obey good amortized bounds in a persistent setting, Okasaki proposes to use laziness. While such programs are especially elegant in a non-strict language like Haskell, Okasaki uses a strict variant of Standard ML in his book. This has the advantage of making it clear where exactly laziness is used in the code and it forces us to give explicit names to the thunks that we will later assign credits to.

We follow this choice and make laziness explicit in our DSL. To do so, we encode Okasaki’s data structures in a monad in call-by-value style [Wadler 1990] and introduce a special delay function to define lazy computations [Peyton Jones and Wadler 1993].

Our class `MonadLazy` uses type families [Chakravarty et al. 2005] to carry an explicit type `Thunk m a` for lazy computations that return values of type `a`. We can delay a computation into a thunk and force a thunk to evaluate it:

```
class Monad m => MonadLazy m where
  data Thunk m :: Type -> Type
  delay :: m a -> m (Thunk m a)
  value :: Thunk m a -> m a
  force :: Thunk m a -> m a
```

The `delay` function puts a computation into a thunk without running it and the `force` function runs the computation stored in a thunk and returns its value. The `value` function is a convenience function to create an evaluated thunk and mirrors the `from_val` function in OCaml. A faithful implementation of the `MonadLazy` class should implement memoization, so that repeated calls to `force` on the same thunk return the same value and without recomputing it.

3.1 Implementing Lazy Computations

As our first instance of `MonadLazy`, we define the `CounterM` monad:

```
data Thunk (CounterM s) a
  = Thunk (STRef s (Either (CounterM s a) a))

newtype CounterM s a
  = CounterM (StateT Ticks (ExceptT String (ST s)) a)
```

```
data Digit a = Zero | One a | Two a a
  deriving (Eq, Ord, Show)

data Implicit a m
  = Shallow (Digit a)
  | Deep (Digit a)
    (Thunk m (Implicit (a, a) m))
    (Digit a)

snoc :: (MonadLazy m, MonadCount m, MonadFail m)
  => Implicit a m -> a -> m (Implicit a m)
snoc q y = do
  tick
  case q of
    Shallow Zero -> do
      pure $ Shallow (One y)
    Shallow (One x) -> do
      m <- value $ Shallow Zero
      pure $ Deep (Two x y) m Zero
    Deep front middle Zero -> do
      pure $ Deep front middle (One y)
    Deep front middle (One x) -> do
      t <- delay $ do
        m <- force middle
        snoc m (x, y)
      pure $ Deep front t Zero
  -> fail "snoc: malformed queue"
```

Fig. 2. The `snoc` function of the Implicit Queue

To implement memoization, we store thunks in an `STRef` using the `ST` monad. We delay computations by storing them as `Left` in the `STRef` and force them by updating the `STRef` to the `Right` value. Additionally, we implement `MonadCount` by keeping an integer state (as the `Ticks` newtype) counting the number of ticks performed by a computation and implement `MonadFail` using the `ExceptT` monad transformer. We can run a computation using:

```
runCounterM :: (forall s. CounterM s a)
  -> Either String (a, Ticks)
```

3.2 The Implicit Queue

As our running example, we will consider the Implicit Queue [Okasaki 1998, Section 11.1]. It is a particularly interesting example, since it is also analysed by the main related works: Danielsson [2008], Xia et al. [2024], and Pottier et al. [2024]. As we will see in the next section, our analysis is closely based on that of Okasaki and can proceed with few modifications to the original code.

We define the Implicit Queue and its `snoc` function in Figure 2. An Implicit Queue is either shallow or deep. A shallow queue contains a digit with up to two elements. A deep queue stores a front digit, a middle thunk, and a rear digit. The middle thunk again contains an Implicit Queue, but instantiates it to store pairs of elements. This is an important trick, since it allows us to recursively store *two* elements in the middle queue at the cost of just *one* recursive call. As the type of elements `a` stored in the Implicit Queue is recursively instantiated, a nested `Deep` node with i nodes above it will contain pairs with 2^i of the original elements each. This trick bounds the depth

of the Implicit Queue (and the worst-case complexity of its operations) to $O(\log n)$.

The `snoc` function adds an element to the back of the queue. We maintain that the right-most digit in the queue only contains zero or one element. As long as the right-most digit is `Zero`, we can simply increment it to `One` and store the new element in it. If the right-most digit is `One` and the queue is shallow, we turn it into a deep queue where we can put both elements in the `front` digit. If we have a deep queue with one element in the `rear` digit, we will call `snoc` recursively to add our new pair of elements into the `middle` queue. As described by Okasaki [1998], we need to make the recursive call to `snoc` lazy to obtain good amortized bounds.

4 Pretty-Printing with Defunctionalized Thunks

Before we discuss how to reason about amortized time complexity, we want to build more intuition for how the Implicit Queue behaves in practice. Unfortunately though, this is not easy when laziness is involved. Lazy computations are inherently opaque and when printing a deep Implicit Queue, we can only observe the top-most digits but not the nested `middle` thunk. This severely limits our ability to build intuitions about the data structure and to debug it.

To gain the ability to inspect thunks, we extend the `MonadLazy` class by user-definable defunctionalized thunks [Lorenzen et al. 2025]. Concretely, we refine the argument of `delay` from an arbitrary computation `m a` to a user-defined abstract data type `t a`. The `delay` function stores `t a` in a thunk, while the `force` function expects an instance of the `HasStep` class, which interprets the type `t a` as a computation `m a`. The `force` function then runs this computation and memoizes the result:

```
class Monad m => MonadLazy m where
  data Thunk m :: (Type -> Type) -> Type -> Type
  delay :: t a -> m (Thunk m t a)
  value :: a -> m (Thunk m t a)
  force :: HasStep t m => Thunk m t a -> m a

class HasStep t m where
  step :: t a -> m a
```

4.1 Defunctionalizing the Implicit Queue

We can use our new interface to defunctionalize the thunks in the Implicit Queue. As is often the case with defunctionalization [Pottier and Gauthier 2004], we need to use GADTs [Cheney and Hinze 2003; Xi et al. 2003] to represent the thunks. An `IThunk m a` corresponds to a defunctionalized computation `m a` in the Implicit Queue:

```
data IThunk m a where
  ISnoc :: Thunk m (IThunk m) (Implicit a m) -> a
  -> IThunk m (Implicit a m)
```

We could mechanically define the `IThunk m a` type by creating one constructor for each thunk in the Implicit Queue

and representing the free variables of the thunk as the children of the constructor. However, we take some liberty here and represent the pair (x, y) as a single child. In the original code (Figure 2), we replace the `delay` call with the `ISnoc` defunctionalized thunk and the final case of `snoc` becomes:

```
Deep front middle (One x) -> do
  t <- delay $ ISnoc middle (x, y)
  pure $ Deep front t Zero
```

Our interpretation step runs exactly the code that we used in the `delay` call previously:

```
instance (MonadLazy m, MonadCount m, MonadFail m)
  => HasStep (IThunk m) m where
  step (ISnoc middle xy) = do
    m <- force middle
    snoc m xy
```

By unfolding the definitions, we can see that this is equivalent to the original code.

4.2 Inspecting Thunks

The main benefit of the defunctionalization above is that it is easily possible to print a defunctionalized thunk. Following Lorenzen et al. [2025], we extend our `MonadLazy` class by a `lazymatch` construct, which gives us internal access to the content of a thunk:

```
class Monad m => MonadLazy m where
  ...
  lazy match
    :: Thunk m t a
    -> ( a -> m b)
    -> (t a -> m b)
    -> m b
```

We can use this to implement functions which pretty-print the data structure in a human-readable way:

```
showImplicit :: (MonadLazy m, Show a)
  => Implicit a m -> m String
showImplicit (Shallow d) = do
  pure $ "(Shallow " ++ show d ++ ")"
showImplicit (Deep f m r) = do
  m' <- showThunk m
  pure $ "(Deep " ++ show f ++ " " ++ m' ++ " "
    ++ show r ++ ")"
```

The magic lies in the `showThunk` function, which can inspect the content of the `middle` thunk. This would be impossible with our previous `MonadLazy` interface, but can now be accomplished using the `lazymatch` construct. If the thunk has been memoized, we can use the `showImplicit` function to print its content. Otherwise, we can pattern-match on the possible lazy thunks and give a human-readable representation of the computation:

```
showThunk :: (MonadLazy m, Show a)
  => Thunk m (IThunk m) (Implicit a m) -> m String
showThunk t = lazy match t showImplicit $ \case
  ISnoc middle xy -> do
    m <- showThunk middle
    pure $ "(snoc " ++ m ++ " " ++ show xy ++ ")"
```

This function allows us now to run and inspect the internal structure of the Implicit Queue:

```
> runCounterM $ (<=<) (traverse (traverse showImplicit)) $
  empty >=> ('snoc' 1) >=> ('snoc' 2) >=> ('snoc' 3)
  >=> ('snoc' 4) >=> ('snoc' 5) >=> uncons
Right (Just (1, "Deep (One 2) (snoc (Shallow Zero) (3,4))
      (One 5)"), Ticks 6)
```

As we can see, the computation succeeds without error (`Right`), performs six steps (`Ticks 6`), and returns the first element of the queue, which is `1`. In the middle thunk, we delay the insertion of the pair `(3, 4)` into an empty queue.

4.3 Recovering the Old Interface

While defunctionalization can yield new insights, it can also obscure the meaning of code. Thankfully, our new interface does not force us to use defunctionalization. Instead, we may instantiate `t` to a `Lazy` type that simply wraps a computation:

```
newtype Lazy m a = Lazy (m a)

instance HasStep (Lazy m) m where
  step (Lazy f) = f
```

This `Lazy` type has the same convenience as our earlier interface, but does not give us the ability to inspect thunks. To save space, we will use the simpler `Lazy` interface from now on. However, our actual implementations use defunctionalized thunks, which has proved extremely useful for debugging.

5 Persistent Amortized Time Complexity

To determine the amortized time complexity of the `Implicit Queue`, we could simply count the number of ticks performed using the `CounterM` monad. We might then observe that on average this number is constant for the `snoc` and `uncons` operations, independently of the size of the queue. This indicates that the `Implicit Queue` has constant amortized time complexity. However, such an approach would yield little insight into *why* that is – after all, this approach is not too different from determining time complexity by measuring the wall clock time of a program.

Instead, with just slightly more work, we can extend our DSL to give us proper invariants. Our approach is informed by Danielsson [2008], who uses a monad `Thunk n a` to model a computation which returns a value of type `a` in at most `n` steps. He then proposes to encode the debit-passing style of Okasaki [1998] by interpreting steps as *debts* which may be transferred between thunks. His primitive:

```
pay : Thunk n a -> Thunk m (Thunk (n - m) a)
```

takes `m` steps and removes `m` debts from the given thunk. That is, it passes debts from the given thunk to the current thunk. In our setting, we want to reason about *credits* and thus go the other way: we pass credits from the current thunk to the given thunk. We encode this principle in the `MonadCredit` class:

```
class (MonadCount m, MonadLazy m, MonadFail m)
  => MonadCredit m where
  creditWith :: Thunk m t a -> Credit -> m ()
  hasAtLeast :: Thunk m t a -> Credit -> m ()
```

The `creditWith` primitive allows us to assign a given amount of credits to a thunk and the `hasAtLeast` primitive asserts that a thunk has at least a given number of credits. Our instance for the `CounterM` monad is trivial and both operations immediately return. But next we will define a more interesting instance.

5.1 The Credit Monad

The `CreditM` monad is a variant of the `CounterM` monad, which tries to run a computation with a number of credits and fails if it runs out. Additionally to the `STRef`, each thunk contains a unique integer to identify it, where the top-level computation of the program is assigned the number `0`. Each thunk has a separate account of credits and a tick executed while evaluating a thunk will deduct one credit from the account of that thunk.

```
data Thunk (CreditM s) t a
  = Thunk !Int !(STRef s (Either (t a) a))

data St = St
{ thunkCurrentlyBeingEvaluated :: Int
, creditsPerThunk :: IntMap Credit }

newtype CreditM s a = CreditM
{ runT :: ExceptT Error (StateT St (ST s)) a }

runCreditM :: Credit -> (forall s. CreditM s a)
  -> Either Error a
```

To be able to force a thunk, we thus need to move credits into the thunk's account. The `creditWith` function allows us to do exactly this: it moves a certain number of credits from the current account into the account of the thunk. Perhaps surprisingly, we can use `creditWith` *inside* a thunk: the credits always come out of the account of the thunk currently being evaluated.

We do not give a soundness proof of this scheme here, but note that it is essentially the same idea as proposed by Danielsson [2008]. One can see that the `creditWith` function does not duplicate credits: all credits that are assigned to an account are taken from the current account, and if the current account is empty, the `CreditM` monad will return an error.

5.2 Implicit Queue

We can prove that the `snoc` function has constant amortized time complexity using the analysis of Okasaki [1998]. Given a deep queue `Deep f m r`, we maintain that `m` has at least `2 - size f + size r` credits and that forcing `m` requires us to supply two credits. The `snoc` function is thus allowed to use two credits: we spend the first credit to perform a tick and use the second credit to enforce the invariant. Let us consider all cases:

- For `q = Shallow (One x)`, we create a deep queue with two elements in the front digit and none in the rear digit. We thus do not have to assign credits to the middle thunk.

```

size :: Digit a -> Credit
size = \case { Zero -> 0; One _ -> 1; Two _ _ -> 2 }

deep f m r = do
  m 'hasAtLeast' (2 - size f + size r)
  pure $ Deep f m r

snoc :: MonadCredit m
      => Implicit a m -> a -> m (Implicit a m)
snoc q y = do
  tick
  case q of
    Shallow Zero -> do
      pure $ Shallow (One y)
    Shallow (One x) -> do
      m <- value $ Shallow Zero
      deep (Two x y) m Zero
    Deep front middle Zero -> do
      middle 'creditWith' 1
      deep front middle (One y)
    Deep front middle (One x) -> do
      t <- delay $ Lazy $ do
        m <- force middle
        snoc m (x, y)
      if size front == 1
      then t 'creditWith' 1
      else middle 'creditWith' 1
      deep front t Zero
    _ -> fail "snoc: malformed queue"

```

Fig. 3. The snoc function of the Implicit Queue with Credits

- For $q = \text{Deep front middle Zero}$, we increase the size of the rear digit by one and thus have to assign one credit to the middle thunk.
- Finally, for $q = \text{Deep front middle (One } x)$, we have to check the front digit. If it has one element, our invariant implies that middle has two credits and is safe to force. But the invariant also states that the new thunk t needs one credit, which we have to assign. Conversely, if front has two elements, then middle has just one credit, and we need to add one to be allowed to force it – but then t does not need additional credit.

In Figure 3, we use the `creditWith` function to assign credits to thunks as indicated by the analysis and the `hasAtLeast` function to check that the invariant holds. The placement of these functions captures the essence of the analysis, while imposing minimal work on the implementor.

5.3 Comparison to Okasaki's Analysis

Our analysis using credit passing style is closely related to Okasaki's analysis in debit passing style. While our main invariant uses credits, it can be derived from Okasaki's debit invariant. In general, debits and credits are related by the equation:

$$\text{debits} = \text{execution cost} - \text{credits}$$

In this case, we can compute our credit invariant above by the execution cost 2 minus Okasaki's debit invariant $\text{size } f - \text{size } r$.

This duality also arises when passing credits and debits: while Okasaki passes debits from the thunk to the enclosing

suspension, we pass credits from the enclosing suspension to the thunk. We believe that this is easier to express algorithmically, since the thunk has a name in the program while the enclosing suspension does not. Furthermore, it seems more intuitive to pass something you have (a credit) rather than something you need (a debit).

When introducing the Bankers Method [Okasaki 1998, Section 6.3], Okasaki defines several notions for the cost of operations. The *unshared* cost is the cost of the operation itself, while the *shared* cost is the cost of all thunks created by the operation, which may be *realized* by the program or remain *unrealized*. These costs can not be expressed in our DSL.

Instead, our DSL models exclusively what Okasaki calls the *amortized* cost of an operation, which is the unshared cost plus the number of debits discharged by the operation – modelled by the `tick` and `creditWith` functions respectively. As Okasaki mentions, this technique gives the correct amortized bounds if we ensure that all debits are discharged by the time a thunk is forced. Using the equation above, this is equivalent to ensuring that the amount of credits on a thunk is at least equal to the execution cost of the thunk.

6 Credit Inheritance

Credit passing style works for almost all data structures in Okasaki's book. Where it does not work, this is because the `creditWith` function needs a name for the thunk to which it passes credits and a precise number of credits to pass.

The Bankers Queue [Okasaki 1998, Section 6.3.2] runs into this problem. It uses (finite) streams to implement a queue, which can be appended lazily. A lazy append operation performs a single unit of work and then creates a thunk for the remaining work. This is a problem for our credit passing style, since the new thunk does not have a name until the append operation is forced, which prevents us from assigning credits to it. Okasaki (as well as the formalizations by Danielsson [2008] and Pottier et al. [2024]) solves this problem by simulating the future behaviour of the append operation and virtually assigning credits to the not-yet-created thunk.

In our approach, we take a different route and use our novel *credit inheritance* principle. Instead of assigning credits to the thunks created by the append operation, we assign all the credits to the top-most thunk of the stream, which passes them to the next thunk when it is forced. While the next thunk has a name in the program, we can not achieve this with the `creditWith` function, since we can not compute the number of credits that are passed. In this section, we introduce the `creditAllTo` primitive to solve this problem.

6.1 Streams

The Bankers Queue relies on streams, which we define in Figure 4. Our stream is similar to a Haskell list, but contains an `SIndirect` constructor which allows us to add thunks to

```

data Stream m a
  = SCons a (Stream m a)
  | SNil
  | SIndirect (Thunk m (Lazy m) (Stream m a))

-- | Smart destructor for streams, consuming one credit
smatch :: MonadCredit m => Stream m a -> ^ Scrutinee
      -> (a -> Stream m a -> m b) -> ^ Cons case
      -> m b -> m b -> ^ Nil case
      -> m b

smatch x cons nil = tick >> eval x
  where
    eval x = case x of
      SCons a as -> cons a as
      SNil -> nil
      SIndirect i -> force i >> eval

taildelay :: MonadCredit m
          => m (Stream m a) -> m (Stream m a)
taildelay t = do
  x <- delay $ Lazy t
  pure (SIndirect x)

ifIndirect (SIndirect i) f = f i
ifIndirect _ _ = pure ()

scredit :: MonadCredit m => Stream m a -> m ()
scredit s = ifIndirect s ('creditWith' 1)

sappend :: MonadCredit m
        => Stream m a -> Stream m a -> m (Stream m a)
sappend xs ys = smatch xs
  (\x xs -> SCons x <$> taildelay (sappend xs ys))
  (pure ys)

sreverse :: MonadCredit m
         => Stream m a -> Stream m a -> m (Stream m a)
sreverse xs ys = smatch xs
  (\x xs -> taildelay $ sreverse xs (SCons x ys))
  (pure ys)

```

Fig. 4. Streams with Inheritance

the stream. We can evaluate those using `smatch`, which recursively forces thunks in the stream until it reaches the weak-head normal form of the stream: either `SNil` or `SCons`. It may be surprising that `smatch` consumes just one credit even if it forces many thunks. However, this corresponds to the cost of matching on a lazy data type in functional languages, where lazy indirections can be short-circuted by the runtime system.

Both `sappend` and `sreverse` create exactly one thunk and call `taildelay` for that. The `taildelay` function creates an indirection for the given computation.

6.2 How to Assign Credits to Streams?

We can use the stream interface to construct a simple stream and then force it fully. Our `walk` function traverses the stream by recursively matching on it:

```
walk s = smatch s (\_ xs -> walk xs) (pure ())
```

As a test, we may append a two-element stream onto another stream. In total, the `sappend` function will perform three matches and thus require three credits to run. Yet, even after

we assign three credits to the front stream, we run out of credits:

```

> runCreditM 3 $ do
  s <- sappend (SCons 1 (SCons 2 SNil)) (SCons 3 SNil)
  scredit s >> scredit s >> scredit s
  walk s
Left (OutOfCredits (Cell 2))

```

Why does this go wrong? It turns out that we have assigned the credits to the wrong thunk. When we walk `s`, all three credits are on the top-most thunk, but that thunk only consumes a single credit (through the `smatch` call). The other two credits are consumed by the *new* thunks created by `sappend`.

We might want to extend `sappend` the same way we extended the `snoc` function of the Implicit Queue: by assigning one credit to the new thunk. However, this can not work: if the `s` thunk passes one credit to the next thunk (`sappend (SCons 2 SNil) (SCons 3 SNil)`), then that thunk will still have just one credit to `smatch` but needs an additional credit to pass on itself.

Danielsson [2008] and Pottier et al. [2024] solve this problem by matching on the thunks created by `sappend` “in the future”. Essentially, by simulating the future behaviour of `sappend`, they are giving names to future thunks which makes it possible to pass credits to them. The same also happens in our analysis of the Realtime Queue [Okasaki 1998, Section 7.2], where the schedule provides a name for the next thunk that needs a credit to be forced. However, for the Bankers Queue this approach is unsatisfying, since it requires us to reason about thunks that do not exist yet.

6.3 Assigning All Credits

We would like to avoid reasoning about future thunks in our program. To achieve this, we take a different approach: instead of assigning a fixed number of credits to a thunk, we design a new primitive, which can assign *all* credits of the current account to a thunk. Assuming we had a primitive `allCredits` that returns the number of credits available in the current account, we might define:

```
creditAllTo x = (x 'creditWith') ==<< allCredits
```

We will use this function in `taildelay` as:

```

taildelay :: MonadCredit m
         => m (Stream m a) -> m (Stream m a)
taildelay t = do
  x <- delay $ Lazy t
  creditAllTo x
  pure (SIndirect x)

```

This fixes our problem: each step of `sappend` consumes one credit and then assigns all remaining credits to the new thunk. If we add three credits to `s`, they will trickle down to the remaining thunks and our computation succeeds.

6.4 Assigning all Future Credits

Unfortunately, the above definition of `creditAllTo` is not quite right. Let us consider what happens if we evaluate the `s` thunk early:

```
> runCreditM 3 $ do
  s <- sappend (SCons 1 (SCons 2 SNil)) (SCons 3 SNil)
  scredit s >> force s >> scredit s >> scredit s
  walk s
Left (OutOfCredits (Cell 2))
```

Again, we run out of credits! The issue is that `creditAllTo` is sensitive to *when* a thunk is evaluated. If we force `s` after assigning three credits to it, it can pass two credits on to the next thunk. But in this example, we force `s` early and it has zero credits left to pass on to the next thunk. Even though we credit `s` twice more, those credits are wasted since they are given to an already-evaluated thunk.

6.5 Credit Inheritance

To solve this problem, we will strengthen the power of `creditAllTo`. Instead of merely passing on all current credits to the new thunk, we will pass on all *future* credits as well. In our example, any credit passed to the already-evaluated thunk `s` will be directly assigned to the next thunk in the stream. The new thunk thus *inherits* all credits of the old thunk, which motivates us to define this in the `MonadInherit` class:

```
class MonadCredit m => MonadInherit m where
  creditAllTo :: Thunk m t a -> m ()
```

To see that this is sound, we can use similar reasoning as for the `MonadCredit` class. In particular, by performing `creditAllTo`, we never duplicate credits. This function only moves credits from the current account to the account of the given thunk (or its heir), which keeps the total number of credits constant.

The `CreditM` monad implements `MonadInherit` by recording the heirs of already-evaluated thunks. When `creditWith` is called on an already-evaluated thunk with an heir, the credits are passed on to the heir. Additionally, we now also keep track of all consumed credits. This allows us to track how many credits were consumed by a thunk before it assigned an heir using `creditAllTo`. The `hasAtLeast` function is unchanged for unevaluated thunks, but for evaluated thunks with an heir, it checks that the heir has at least the given number of credits minus the number of credits consumed by the evaluated thunk. This ensures that `hasAtLeast` provides the same guarantee for evaluated thunks as it does for unevaluated thunks.

6.6 Bankers Queue

The `MonadInherit` class allows us to give a short proof that Okasaki's Bankers Queue has constant amortized time complexity for both `snoc` and `uncons`. The main credit invariant of the Bankers Queue is:

- The rear stream has length `rlen` and is always fully evaluated.
- The front stream has length `flen` and is either fully evaluated or a thunk of the form `sappend xs ys`, which has `rlen`-many credits assigned to it. In that case, the thunks in the left child `xs` require no further credits,

```
data BQueue a m = BQueue
{ front :: Stream m a
, flen :: Int
, rear :: Stream m a
, rlen :: Int
}

bqueue :: MonadInherit m => BQueue a m -> m (BQueue a m)
bqueue (BQueue f fl r rl) = do
  ifIndirect f ('hasAtLeast' fromIntegral rl)
  if fl >= rl
  then pure $ BQueue f fl r rl
  else do
    r' <- delay $ Lazy $ sreverse r SNil
    r' <- 'creditWith' 1
    f' <- delay $ Lazy $ sappend f (SIndirect r')
    pure $ BQueue (SIndirect f') (fl + rl) SNil 0

snoc :: MonadInherit m => BQueue a m -> a -> m (BQueue a m)
snoc (BQueue f fl r rl) x = do
  scredit f
  bqueue $ BQueue f fl (SCons x r) (rl + 1)

uncons :: MonadCount m
=> BQueue a m -> m (Maybe (a, BQueue a m))
uncons (BQueue f fl r rl) = do
  scredit f >> scredit f
  smatch f
  (\x f -> do
    q <- bqueue (BQueue f (fl - 1) r rl)
    pure $ Just (x, q))
  (pure Nothing)
```

Fig. 5. Bankers Queue based on streams

but `ys` is a `sreverse` thunk which has no credits and requires `length(xs)`-many credits.

This invariant allows us to use the `bqueue` operation. Once `rlen > flen`, we have more than `flen`-many credits on the top-most `sappend` thunk. Since `flen >= 2 * length(xs)`, this is enough to perform the remaining appends and pay off the `sreverse` thunk. The new `sreverse` thunk requires `length(r)` many credits, but since `bqueue` only gets called with queues where `rlen <= flen + 1`, we may assign one credit to `sreverse` to obtain the invariant.

The `snoc` operation maintains this invariant since it increases `rlen` by one and assigns one credit to the `front` stream. The `uncons` operation might have to force the topmost `sappend` thunk and assigns one credit to it. Additionally, this reduces the length of `xs` by one, which means `uncons` needs to assign one credit to the `sreverse` thunk. Instead of assigning this credit in `uncons` though, we can perform this as part of `sappend`, where we now write:

```
sappend xs ys = scredit ys >> smatch xs ...
```

Since the second argument of `sappend` will always be an indirection to a `sreverse` thunk, this assigns the credit correctly.

The Bankers Deque is closely related to the Bankers Queue and can be analyzed in a similar way.

```

data Digit a = One a | Two a a | Three a a a
  deriving (Eq, Ord, Show)

data Tuple v a = Pair v a a | Triple v a a a
  deriving (Eq, Ord, Show)

data FingerTree v a m
  = Empty
  | Single a
  | Deep v (Digit a)
    (Thunk m (Lazy m) (FingerTree v (Tuple v a) m))
    (Digit a)

isTwo :: Digit a -> Bool
isTwo (Two _ _) = True
isTwo _ = False

```

Fig. 6. Definition of Finger Trees

6.7 Remaining Challenges

Catenable Lists [Okasaki 1998, Section 10.2.1] probably have the hardest analysis in Okasaki’s book. Not only do they require the novel analysis technique of a left-linear debit invariant, but it is not even clearly specified which thunks credits are assigned to: Okasaki merely writes that “Discharging the first two debits restores the invariant”. In our implementation, we thus perform a depth-first traversal of the catenable list and assign credits to the first two thunks we encounter.

However, this is a deeply unsatisfying solution. We have to force the thunks to avoid assigning credits to the same thunk multiple times; this is unlike all other data structures which we can analyse without changing the original program. Even worse, our code provides no insight as to why catenable lists have the correct amortized time complexity. We leave the search for a better analysis as future work.

The Implicit Catenable Deque [Okasaki 1998, Section 11.2] is a version of the Implicit Queue that allows for $O(1)$ concatenation. However, our tests currently do not cover the case where two deep dequeues are concatenated and we are not sure if the argument continues to hold in this case. The issue arises in the case where a is empty and the head of b is a compound element. Since the `tail` operation performs $O(1)$ work and also discharges five debits, this suggests that the suspension for `$tail (force b)` has six debits. However, Okasaki’s argument assumes that this suspension has at most five debits, which does not seem to be correct (or easily fixable). We hope to investigate this further in the future.

7 Case Study: Finger Trees

Finger Trees [Hinze and Paterson 2006] are one of the most versatile data structures available to functional programmers: they possess both theoretical elegance and practical utility (as Haskell’s `Data.Sequence` library).

Finger Trees can be described as an extension of the Implicit Queue, that also allows for an efficient `concat` operation. Assume we were to concatenate the Implicit Queue

```

snoc :: (MonadCredit m, Measured a v)
  => FingerTree v a m -> a -> m (FingerTree v a m)
snoc q w = do
  tick
  case q of
    Empty -> pure $ Single w
    Single x -> do
      em <- value Empty
      deep (One x) em (One w)
    Deep v front middle (One x) ->
      deep front middle (Two x w)
    Deep v front middle (Two x y) -> do
      middle 'creditWith' 1
      deep front middle (Three x y w)
    Deep v front middle (Three x y z) -> do
      t <- delay $ Lazy $ do
        m <- force middle
        snoc m (pair x y)
      if isTwo front
      then middle 'creditWith' 1
      else t
      deep front t (Two z w)

```

Fig. 7. The `snoc` function of Finger Trees (without measurement)

`Deep f1 m1 (One x)` with `Deep (Two y z) m2 r2`. We would like to return `Deep f1 (m1 ++ (x, y, z) ++ m2) r2`, but since the Implicit Queue only stores tuples of size two, we can not insert three elements in the middle without invasive restructuring. Finger Trees address this problem by storing tuples of both size 2 and size 3 in the middle tree.

In our implementation, we use the simplified implementation of Claessen [2020] although we sometimes prefer the constructor names of Hinze and Paterson [2006], since they better match the Implicit Queue.

Given their similarities, it will not be surprising that Finger Trees can be analyzed very similarly to the Implicit Queue. The `cons`, `snoc`, `tail`, and `init` operations all cost two credits, where the first credit is used to perform a `tick` and the second credit is used to maintain the credit invariant:

```

deep f m r = do
  let dang d = if isTwo d then 0 else 1
  m 'hasAtLeast' (dang f + dang r)
  pure $ Deep f m r

```

In each queue `Deep f m r`, the m thunk has at least one credit for each digit among f and r that is not a `Two` digit. This `dang` invariant also occurs in the sequential analysis given by Claessen [2020].

In the `snoc` function we can see the similarity to the Implicit Queue. Considering the `Deep` cases in the `snoc` function, we can analyse:

- **One:** Here we turn a `One` digit into a `Two` digit, which even reduces the number of credits required by the middle tree by one and thus requires no additional credits.
- **Two:** Here we turn a `Two` digit into a `Three` digit and thus require one additional credit for the middle tree.
- **Three:** Again, we case-split on whether the front digit is a `Two`. If this is the case, we need to supply one more credit to the middle tree to make sure that it has the two credits required to be able to force it. Otherwise, we

have to give this credit to the new t thunk to maintain the invariant.

7.1 Measurement

One of the key selling points of Finger Trees is that they can be parameterized by a monoid v . By choosing different instances of this monoid, we can turn Finger Trees into different data structures, such as heaps, random-access lists, or even interval trees. The class `Measured` allows us to map the elements of a Finger Tree to the monoid v :

```
class Monoid v => Measured a v where
  measure :: a -> v

instance Monoid v => Measured (Tuple v a) v where
  measure (Pair v _ _) = v
  measure (Triple v _ _ _) = v
```

By storing the measurement in the tuples, Hinze and Paterson [2006] ensure that the measurement of elements can be accessed in constant time at all levels of the tree. We can even measure the Finger Tree itself in constant time, if we cache the measurement in the `Deep` nodes. While Hinze and Paterson [2006] propose to store the combined measurement of the prefix, middle tree and suffix, we store the measurement of the middle tree only:

```
instance Measured a v
  => Measured (FingerTree v a m) v where
  measure Empty = mempty
  measure (Single x) = measure x
  measure (Deep v f m r) = measure f <> v <> measure r
```

All operations of Finger Trees that change the middle thunk now have to recompute the cached measurement. Hinze and Paterson [2006] suggest to extend the smart constructor `deep` to recompute the measurement:

```
deep f m r = do
  m' <- force m
  pure $ Deep (measure m') f m r
```

Yet, in this implementation, we actually force the thunks of the middle tree. If we were to use this implementation in a persistent setting, this would break the amortized time complexity of Finger Trees: By forcing m whenever a `Deep` node is constructed, we can no longer keep unevaluated thunks in the middle tree.

Instead, we have to be a bit smarter and compute the measurement of the middle tree without forcing it. We thus add v as an additional argument to the `deep` function and keep the existing cached measurement when the middle tree stays the same. The only hard case in the `snoc` function occurs when we recursively add (x, y) to middle: then we have to add the measurement of (x, y) to the cached measurement v . The last line of listing 7 becomes:

```
deep (v <> measure (x, y)) front t (Two z w)
```

But while this is easy for `snoc` where we add elements, it is more difficult for `tail`, where we remove elements from the middle tree. Adding elements can easily be implemented by appending the measurement. But removing elements is harder, since monoids do not support subtraction.

```
tail :: (MonadCredit m, Measured a v)
  => FingerTree v a m -> m (FingerTree v a m)
tail q = do
  tick
  case q of
    Empty -> pure Empty
    Single _ -> pure Empty
    Deep v (Three _ x y) m r -> do
      deep v (Two x y) m r
    Deep v (Two _ x) m r -> do
      m 'creditWith' 1
      deep v (One x) m r
    Deep _ (One _) m r -> do
      when (isTwo r) $ m 'creditWith' 1
      force m >>= ('deep0' r)

deep0 :: (MonadCredit m, Measured a v)
  => FingerTree v (Tuple v a) m -> Digit a
  -> m (FingerTree v a m)
deep0 Empty r = toTree $ toList r
deep0 m r = do
  hd <- head m
  case hd of
    Pair _ x y -> do
      t <- delay $ Lazy $ tail m
      unless (isTwo r) $ t 'creditWith' 1
      let v = measureTail m
      deep v (Two x y) t r
    Triple _ x _ _ -> do
      let m' = map1 chop m
      m'' <- value m'
      deep (measure m') (One x) m'' r
  where chop (Triple _ _ y z) = pair y z

map1 :: (Measured a v) => (a -> a)
  -> FingerTree v a m -> FingerTree v a m
map1 f q = case q of
  Empty -> Empty
  Single x -> Single (f x)
  Deep v (One x) m r -> Deep v (One (f x)) m r
  Deep v (Two x y) m r -> Deep v (Two (f x) y) m r
  Deep v (Three x y z) m r -> Deep v (Three (f x) y z) m r
```

Fig. 8. The tail function of Finger Trees

7.2 Tail with Constant Time Measurement

In Figure 8 we present the `tail` function of Finger Trees. Again, our presentation follows Claessen [2020] with few annotations. Just like in their presentation, our `deep0` function pattern-matches on the queue to handle the empty case differently. This requires us to force the middle tree before calling `deep0` and thus split our credit assignment into two parts: if r is a `Two` digit, we need to assign one credit to the middle tree before forcing it, otherwise we assign one credit to the new thunk t .

Finally, how do we implement the measurement of the tail thunk of a Finger Tree? Here, we need an insight suggested to the author by Ross Paterson in a private communication and implemented in Haskell's `fingertree` package. The key idea is to use the cached measurement of the tree one level below. The `measureTail` function below computes the measurement of the tail thunk by using the fact that `tail` removes the first element:

```

measureTail :: Measured a v
             => FingerTree v (Tuple v a) m -> v
measureTail q = case q of
  Empty -> mempty
  Single _ -> mempty
  Deep v f _ r -> case f of
    One _ -> v <> measure r
    Two _ y -> measure y <> v <> measure r
    Three _ y z -> measure (y, z) <> v <> measure r

```

It is easy to see that this function is correct in all cases where tail does not recurse. If tail recurses (i.e., the front digit is *One*), it has to perform extra rebalancing steps – but this rebalancing does not change the measurement of the tree.

8 Related Work

Okasaki [1998] was the first to give amortized time bounds to data structures in a persistent setting and pioneered the use of lazy evaluation to enable this. Subsequent work mostly focussed on improving the style of analysis itself.

Debit-Style Reasoning. Danielsson [2008] gave the first mechanization of Okasaki’s debit-passing style. He verifies the Implicit Queue, the Bankers Queue and the deque operations of Finger Trees in Agda. His analysis of the Implicit Queue uses a slightly more complicated invariant than Okasaki’s, but he uses Okasaki’s invariant in the code accompanying his paper. His use of a *Thunk* monad to represent both thunks and computations with credits enables the elegant type signature of *pay*, but is also somewhat confusing since it obscures where thunks are created.

Pottier et al. [2024] define an Iris logic for reasoning about debits based on time credits [Mével et al. 2019]. They verify the Bankers Queue, Physicists Queue and Implicit Queue. Their proofs are similar to those of Danielsson [2008], but their proof rules are more general and lead to more modular proofs. In particular, all of their reasoning happens outside of the actual program; this makes it clear where thunks are created and that the verified program corresponds to the original program.

Lazy Physicists Method. Hackett and Hutton [2019] note that to reason about time complexity, it does not matter *when* a thunk is evaluated, but only *whether* it is evaluated at all. This makes it possible to use demand analysis to reason about them [Bjerner and Holmström 1989; Foner et al. 2018; Li et al. 2021]. Xia et al. [2024] use this insight to analyze the Bankers Queue and the Implicit Queue. Their method is very promising, but also significantly different from ours, which makes a comparison difficult. We would expect their technique to be preferable to ours when reasoning about implementations in a lazy language, where a high number of thunks is created implicitly. However, we believe that our approach is nicer for reasoning about lazy data structures in strict languages, in particular if the number of thunks is small as in Okasaki’s work.

Type Theories. Jost et al. [2017] describe a type theory for estimating the runtime cost of lazy computations. They annotate function arrows and thunks by cost annotations that give the number of steps that have to be performed to run the function or evaluate the thunk. Similar to Danielsson [2008]’s *pay* primitive, they have a *PREPAY* rule which allows us to reduce the cost annotation of a thunk by paying for it early. They do not apply their system to Okasaki’s data structures.

Madhavan et al. [2017] describe a tool for estimating costs of functions with memoization. They use defunctionalization to transform higher-order programs into first-order form and use a solver to find a cost bound. They apply their system to Okasaki’s real-time queue and deque, where their system finds the correct constant-time bounds with a very good upper bound on the number of steps required.

Finger Trees. Finger Trees have been formally verified before, but we seem to be the first to analyse all of the deque operations, concatenation and the *splitTree* function in a persistent setting. Danielsson [2008] verifies the deque operations of Finger Trees in a persistent setting. Claessen [2020] uses QuickCheck to test the sequential time complexity of the deque operations and concatenation, but leaves the persistent amortized time complexity of their simplified Finger Trees as future work. Brügge [2024] verifies sequential Finger Trees in Liquid Haskell.

9 Future Work

We do not provide a formal proof of soundness for credit passing and credit inheritance. This is acceptable for our purposes since they only aid in testing, but would be crucial to building a formally verified library of amortized data structures.

The techniques of credit passing and credit inheritance are quite general, but do not work well for Catenable Lists. We are interested in finding out if it is possible to amend Okasaki’s proof in a way that allows us to employ credit passing, or if this points to a more fundamental limitation of our reasoning principles.

Acknowledgements

Francois Pottier pointed out that we do not test all persistent use-cases and suggested the use of an assertion to check the number of credits. Wouter Swierstra, Wenhao Tang and Sam Lindley suggested improvements to the presentation of the paper.

References

- Clément Allain, Basile Clément, Alexandre Moine, and Gabriel Scherer. “Snapshottable Stores.” *Proceedings of the ACM on Programming Languages* 8 (ICFP). ACM New York, NY, USA: 338–369. 2024. doi:[10.1145/3674637](https://doi.org/10.1145/3674637).
- Bror Bjerner, and Sören Holmström. “A Composition Approach to Time Analysis of First Order Lazy Functional Programs.” In *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*, 157–165. 1989. doi:[10.1145/99370.99382](https://doi.org/10.1145/99370.99382).
- Jan van Brügge. “Liquid Amortization: Proving Amortized Complexity with LiquidHaskell (Functional Pearl).” In *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium*, 97–108. 2024. doi:[10.1145/3677999.3678282](https://doi.org/10.1145/3677999.3678282).
- Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. “Associated Types with Class.” In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1–13. 2005. doi:[10.1145/1040305.1040306](https://doi.org/10.1145/1040305.1040306).
- James Cheney, and Ralf Hinze. *First-Class Phantom Types*. Cornell University. 2003.
- Koen Claessen. “Finger Trees Explained Anew, and Slightly Simplified (functional Pearl).” In *Proceedings of the 13th ACM SIGPLAN International Symposium on Haskell*, 31–38. 2020. doi:[10.1145/3406088.3409026](https://doi.org/10.1145/3406088.3409026).
- Koen Claessen, and John Hughes. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–279. 2000. doi:[10.1145/351240.351266](https://doi.org/10.1145/351240.351266).
- Nils Anders Danielsson. “Lightweight Semiformal Time Complexity Analysis for Purely Functional Data Structures.” *ACM SIGPLAN Notices* 43 (1). ACM New York, NY, USA: 133–144. 2008. doi:[10.1145/1328438.1328457](https://doi.org/10.1145/1328438.1328457).
- Kenneth Foner, Hengchu Zhang, and Leonidas Lampropoulos. “Keep Your Laziness in Check.” *Proceedings of the ACM on Programming Languages* 2 (ICFP). ACM New York, NY, USA: 1–30. 2018. doi:[10.1145/3236797](https://doi.org/10.1145/3236797).
- Jeremy Gibbons, and Ralf Hinze. “Just Do It: Simple Monadic Equational Reasoning.” *ACM SIGPLAN Notices* 46 (9). ACM New York, NY, USA: 2–14. 2011. doi:[10.1145/2034773.2034777](https://doi.org/10.1145/2034773.2034777).
- Jennifer Hackett, and Graham Hutton. “Call-by-Need Is Clairvoyant Call-by-Value.” *Proceedings of the ACM on Programming Languages* 3 (ICFP). ACM New York, NY, USA: 1–23. 2019. doi:[10.1145/3341718](https://doi.org/10.1145/3341718).
- Ralf Hinze, and Ross Paterson. “Finger Trees: A Simple General-Purpose Data Structure.” *Journal of Functional Programming* 16 (2). Cambridge University Press: 197–217. 2006. doi:[10.1017/S0956796805005769](https://doi.org/10.1017/S0956796805005769).
- Steffen Jost, Pedro Vasconcelos, Mário Florido, and Kevin Hammond. “Type-Based Cost Analysis for Lazy Functional Languages.” *Journal of Automated Reasoning* 59 (1): 87–120. 2017. doi:[10.1007/s10817-016-9398-9](https://doi.org/10.1007/s10817-016-9398-9).
- Haim Kaplan, Chris Okasaki, and Robert E Tarjan. “Simple Confluently Persistent Catenable Lists.” *SIAM Journal on Computing* 30 (3). SIAM: 965–977. 2000. doi:[10.1007/BFb0054360](https://doi.org/10.1007/BFb0054360).
- Yao Li, Li-yao Xia, and Stephanie Weirich. “Reasoning about the Garden of Forking Paths.” *Proceedings of the ACM on Programming Languages* 5 (ICFP). ACM New York, NY, USA: 1–28. 2021. doi:[10.1145/3473585](https://doi.org/10.1145/3473585).
- Anton Lorenzen, Daan Leijen, Wouter Swierstra, and Sam Lindley. “First-Order Laziness.” *Proceedings of the ACM on Programming Languages*, number ICFP. ACM New York, NY, USA. 2025. doi:[10.1145/3747530](https://doi.org/10.1145/3747530).
- Ravichandhran Madhavan, Sumith Kulal, and Viktor Kuncak. “Contract-Based Resource Verification for Higher-Order Functions with Memoization.” In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, 330–343. POPL ’17. Association for Computing Machinery, New York, NY, USA. 2017. doi:[10.1145/3009837.3009874](https://doi.org/10.1145/3009837.3009874).
- Glen Mével, Jacques-Henri Jourdan, and François Pottier. “Time Credits and Time Receipts in Iris.” In *Programming Languages and Systems: 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6–11, 2019, Proceedings* 28, 3–29. Springer. 2019. doi:[10.1007/978-3-030-17184-1_1](https://doi.org/10.1007/978-3-030-17184-1_1).
- Chris Okasaki. *Purely Functional Data Structures*. Cambridge University Press. 1998.
- Simon L Peyton Jones, and Philip Wadler. “Imperative Functional Programming.” In *Proceedings of the 20th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 71–84. 1993. doi:[10.1145/158511.158524](https://doi.org/10.1145/158511.158524).
- François Pottier. “Strong Automated Testing of OCaml Libraries.” In *JFLA 2021 - 32es Journées Francophones Des Langues Applicatifs*. Saint Médard d’Excideuil, France. Feb. 2021. <https://inria.hal.science/hal-03049511>.
- François Pottier, and Nadji Gauthier. “Polymorphic Typed Defunctionalization.” ACM New York, NY, USA, 89–98. 2004. doi:[10.1145/964001.964009](https://doi.org/10.1145/964001.964009).
- François Pottier, Armaël Guéneau, Jacques-Henri Jourdan, and Glen Mével. “Thunks and Debits in Separation Logic with Time Credits.” *Proceedings of the ACM on Programming Languages* 8 (POPL). ACM New York, NY, USA: 1482–1508. 2024. doi:[10.1145/3632892](https://doi.org/10.1145/3632892).
- Philip Wadler. “Comprehending Monads.” In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming*, 61–78. 1990. doi:[10.1145/91556.91592](https://doi.org/10.1145/91556.91592).
- Li-yao Xia, Laura Israel, Maite Kramarz, Nicholas Coltharp, Koen Claessen, Stephanie Weirich, and Yao Li. “Story of Your Lazy Function’s Life: A Bidirectional Demand Semantics for Mechanized Cost Analysis of Lazy Programs.” *Proc. ACM Program. Lang.* 8 (ICFP). Association for Computing Machinery, New York, NY, USA. Aug. 2024. doi:[10.1145/3674626](https://doi.org/10.1145/3674626).
- Hongwei Xi, Chiyan Chen, and Gang Chen. “Guarded Recursive Datatype Constructors.” In *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 224–235. 2003. doi:[10.1145/604131.604150](https://doi.org/10.1145/604131.604150).